

Instructor: Bernal Manzanilla Saavedra

IT 213: Networking Lab 10: The Link Layer – Ethernet, MAC Addresses, and ARP

Objective: In Chapter 6, we learned that the Network Layer relies on the Link Layer to physically move frames across a local network. In this lab, you will use Wireshark on your local machine to observe Address Resolution Protocol (ARP) broadcasts, and you will use a Google Colab Linux terminal environment to inspect MAC addresses and local ARP tables.

Environment Setup:

1. **Local Machine:** Open Wireshark.
 2. **Terminal Simulator:** Open Google Colab (colab.research.google.com), create a New Notebook, and use the `!` prefix to run Linux terminal commands.
-

Part 1: Inspecting Network Interfaces (Colab Terminal) Every network interface controller (NIC) has a permanently burned-in 48-bit MAC address. Let's find the MAC address of our cloud Linux instance.

1. In your Colab notebook, run the following command to show link-layer details: `!ip link show`
2. **Lab Report Question 1:** Look at the output for the `eth0` interface. What is the 48-bit MAC address (listed after `link/ether`)?
3. **Lab Report Question 2:** Notice the `lo` (loopback) interface. What is its MAC address state, and why does a loopback interface not need a standard hardware MAC address?

Part 2: The Address Resolution Protocol (Colab Terminal) When a host needs to send an IP datagram to a local device, it must translate the IP address to a MAC address using ARP.

1. In a new Colab cell, check the current ARP table (which caches IP-to-MAC mappings): `!arp -a`
2. **Lab Report Question 3:** What IP addresses and corresponding MAC addresses are currently cached in your Colab instance's ARP table?
3. To see how ARP actively resolves an address, let's ping the Google public DNS. Run: `!ping -c 3 8.8.8.8`
4. Now, check the ARP table again: `!arp -a`
5. **Lab Report Question 4:** Did the MAC address for `8.8.8.8` appear in your ARP table? Why or why not? (*Hint: Re-read Section 6.4.1. Does ARP resolve addresses outside the local subnet, or does it resolve the MAC of the default gateway?*)

Part 3: Capturing ARP in Action (Wireshark) Now we will observe the actual ARP broadcast frames on your local physical network.

1. Open Wireshark and begin capturing on your active interface (Wi-Fi or Ethernet).
2. Leave the display filter blank for a moment.
3. Open your computer's local Command Prompt / Terminal and clear your ARP cache to force a new broadcast:
 - *Windows (Run as Admin):* `arp -d *`
 - *Mac:* `sudo arp -a -d`
 - *Linux:* `sudo ip neigh flush all`
4. Ping your default gateway (your home/campus router). Example: `ping 192.168.1.1`
5. Stop the Wireshark capture. In the display filter, type `arp` and press Enter.
6. **Lab Report Question 5:** Find the "ARP Request" packet (e.g., "Who has 192.168.1.1?"). Expand the **Ethernet II** header. What is the Destination MAC address of this frame? Explain why this specific address (all F's) is used.
7. **Lab Report Question 6:** Find the "ARP Reply" packet. Expand the **Ethernet II** header. What are the Source and Destination MAC addresses? How do these compare to the Request frame?